BỘ GIÁO DỤC VÀ ĐÀO TẠO - HỆ LIÊN THÔNG

BÁO CÁO BÀI TẬP THỰC HÀNH HỌC PHẦN LẬP TRÌNH BACKEND

CHƯƠNG 06: AUTHENTICATION & AUTHORIZATION

JWT - MIDDLEWARE KIỂM SOÁT - PHÂN QUYỀN VAI TRÒ (RBAC) - BUỔI 07

- **Họ và tên sinh viên:** **ĐẶNG BẮC NAM**
- **Mã sinh viên / Dự án:** BACKEND-20261001-DANGBACNAM

- **Công nghệ nền tảng:** Node.js (v20+), Express 5, TypeScript ESM, Sequelize ORM 6, SQL Server, JWT (HS256), bcrypt (cost 12), Swagger OpenAPI 3.0
- **GitHub Repository:** https://github.com/425000218/BACKEND-20261001-DANGBACNAM.git
- **Kết quả kiểm thử:** **ĐẠT 24/24 ca kiểm thử tự động (Tỷ lệ: 100.0%)**
- **Thời gian hoàn thành:** Tháng 10/2026

1. Đề bài, Mục tiêu và Hai lớp kiểm soát bắt buộc

Hệ thống Quản lý sinh viên sau Chương 05 đã có đầy đủ 5 API CRUD, Validation và Swagger UI nhưng chưa có lớp bảo vệ tài khoản. Mục tiêu của Chương 06 là nâng cấp hệ thống để quản lý tài khoản người dùng, cấp Access Token JWT khi đăng nhập, từ đó nhận diện người gửi Request và phân quyền RBAC: chỉ cho phép Quản trị viên (admin) thực hiện thêm, sửa, xóa sinh viên; người dùng thông thường (student) chỉ được xem danh sách/chi tiết công khai và hồ sơ chính mình.

Bảng Hợp đồng API (API Contracts)

Method & Endpoint	Phân quyền	Mô tả nghiệp vụ	Mã Status
POST /api/auth/register	Public	Đăng ký tài khoản mới, role mặc định là student	201 / 400 / 409
POST /api/auth/login	Public	Đăng nhập xác thực, cấp Access Token JWT (1h)	200 / 401
GET /api/auth/profile	**isAuth**	Xem thông tin hồ sơ của tài khoản đang đăng nhập	200 / 401
GET /api/v1/sinh-vien	Public	Lấy danh sách toàn bộ sinh viên trong CSDL	200

GET /api/v1/sinh-vien/:maSV	Public	Xem chi tiết một sinh viên theo mã	200 / 404
POST /api/v1/sinh-vien	**isAuth + isAdmin**	Thêm mới sinh viên vào CSDL SQL Server	201 / 401 / 403
PUT /api/v1/sinh-vien/:maSV	**isAuth + isAdmin**	Cập nhật thông tin sinh viên theo mã	200 / 401 / 403
DELETE /api/v1/sinh-vien/:maSV	**isAuth + isAdmin**	Xóa sinh viên khỏi CSDL SQL Server	200 / 401 / 403

• **Nguyên tắc phân biệt mã HTTP:** Mã 401 Unauthorized đại diện cho lỗi xác thực danh tính (thiếu token, token sai chữ ký hoặc hết hạn); Mã 403 Forbidden đại diện cho lỗi phân quyền (đã xác thực danh tính thành công nhưng tài khoản sinh viên không có quyền ghi). Bắt buộc phải gắn isAuth trước isAdmin.

2. Thiết kế Cơ sở dữ liệu SQL Server & Sequelize Model

Để lưu trữ tài khoản người dùng bền vững, dự án thiết kế bảng dbo.Users trong SQL Server với script sql/04-users-auth.sql, đảm bảo tính idempotent (chạy lại nhiều lần không phá hỏng cấu trúc cũ) và cấp quyền cho tài khoản ứng dụng sv_app:

Tên trường	Kiểu dữ liệu SQL	Ràng buộc & Khóa	Ý nghĩa & Quy tắc an toàn
id	INT	IDENTITY(1,1), PK	Khóa chính tự tăng, định danh người dùng trong token (sub)
fullName	NVARCHAR(150)	NOT NULL	Họ tên người dùng tiếng Việt có dấu
email	NVARCHAR(254)	NOT NULL, UNIQUE	Email đăng nhập duy nhất, chống trùng lặp tài khoản
passwordHash	NVARCHAR(255)	NOT NULL	Chuỗi băm bcrypt ($2b$12$...), tuyệt đối không lưu plaintext
role	NVARCHAR(20)	DEFAULT 'student'	Vai trò người dùng ('student' hoặc 'admin')

• **Phân quyền tối thiểu cho sv_app:** GRANT SELECT, INSERT, UPDATE, DELETE ON dbo.Users TO sv_app. Đảm bảo Backend có đủ quyền tra cứu lúc login, chèn lúc register và đồng bộ tài khoản admin dev.

3. Triển khai Kỹ thuật & Mã nguồn cốt lõi

3.1. Băm mật khẩu với bcrypt (Salt cost 12): Khi đăng ký (Register), gọi bcrypt.hash(password, 12) sinh chuỗi hash kèm muối ngẫu nhiên. Khi đăng nhập (Login), bắt buộc gọi bcrypt.compare(password, user.passwordHash) để trích xuất salt từ hash có sẵn và so khớp, chống tấn công đo thời gian (Timing Attack).

3.2. Tiện ích JWT Utility (src/utils/jwt.ts): Hàm generateToken ký chuỗi JWT bằng thuật toán HS256, khóa bí mật JWT_SECRET đọc từ .env, thời hạn sống đúng 1 giờ ({ expiresIn: "1h", algorithm: "HS256" }). Hàm verifyToken xác minh chữ ký, hạn dùng và thuật toán, bắt các lỗi TokenExpiredError và JsonWebTokenError.

3.3. Middleware Xác thực isAuth (src/middlewares/isAuth.ts): Trích xuất header Authorization, kiểm tra định dạng "Bearer <token>". Giải mã token hợp lệ và gán req.user = { id: payload.sub, role: payload.role }. Nếu thiếu header, sai định dạng, token hết hạn hoặc sai chữ ký, lập tức phản hồi HTTP 401 Unauthorized.

3.4. Middleware Phân quyền isAdmin (src/middlewares/isAdmin.ts): Kiểm tra req.user.role === "admin". Nếu là vai trò "student", lập tức chặn lại và phản hồi HTTP 403 Forbidden kèm thông báo: "Yêu cầu quyền Quản trị viên (admin)".

3.5. Mở rộng TypeScript (src/types/express.d.ts): Mở rộng interface Express.Request để chứa thuộc tính user?: IUserAuth, giúp compiler TypeScript nhận diện req.user một cách an toàn mà không báo lỗi Type.

3.6. Script Khởi tạo Admin (src/scripts/seed-admin.ts): Tạo sẵn tài khoản Admin môi trường Dev (admin@example.com / Admin@123) với role "admin" trực tiếp trong CSDL SQL Server, vì hệ thống từ chối cho phép client tự đăng ký role admin qua Public API (chống leo quyền).

4. Kết quả Nghiệm thu Ma trận 24 Ca Kiểm thử Tự động

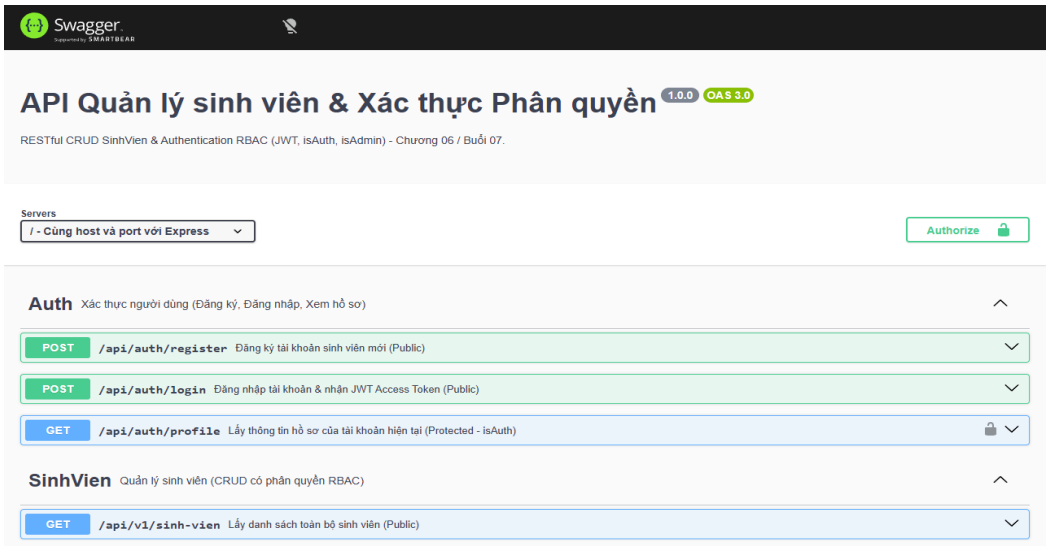
Toàn bộ 24 ca test (Nhóm A: A01–A12 cho Auth/JWT; Nhóm B: B01–B12 cho RBAC SinhVien) được thực thi tự động qua test runner pnpm test:24. Kết quả đạt tỷ lệ tuyệt đối 100.0% (24/24 ca ĐẠT):

Mã Ca	Tên Ca Kiểm Thử	Method & URL	Kỳ vọng / Thực tế	Kết quả
A01	Đăng ký tài khoản mới hợp lệ	POST /api/auth/register	201 / 201	**[ĐẠT] PASS**
A02	Đăng ký với email đã tồn tại	POST /api/auth/register	409 / 409	**[ĐẠT] PASS**
A03	Đăng ký kèm role admin (leo quyền)	POST /api/auth/register	400 / 400	**[ĐẠT] PASS**
A04	Đăng nhập thông tin chính xác	POST /api/auth/login	200 / 200	**[ĐẠT] PASS**

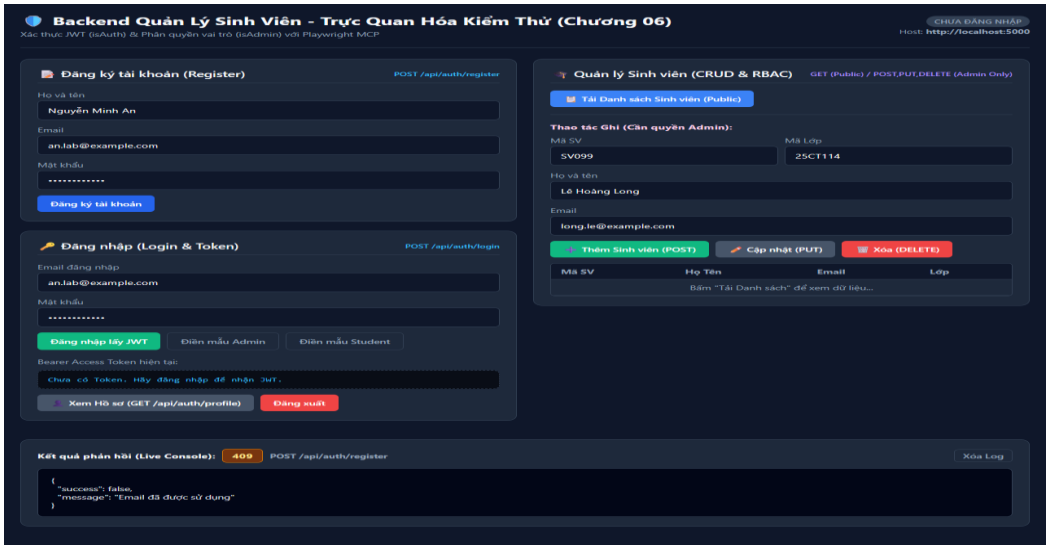
A05	Đăng nhập sai mật khẩu	POST /api/auth/login	401 / 401	**[ĐẠT] PASS**
A06	Đăng nhập email không tồn tại	POST /api/auth/login	401 / 401	**[ĐẠT] PASS**
A07	Xem hồ sơ không gửi Authorization	GET /api/auth/profile	401 / 401	**[ĐẠT] PASS**
A08	Xem hồ sơ sai định dạng Bearer	GET /api/auth/profile	401 / 401	**[ĐẠT] PASS**
A09	Xem hồ sơ Token giả mạo chữ ký	GET /api/auth/profile	401 / 401	**[ĐẠT] PASS**
A10	Xem hồ sơ Token đã hết hạn	GET /api/auth/profile	401 / 401	**[ĐẠT] PASS**
A11	Xem hồ sơ Token hợp lệ	GET /api/auth/profile	200 / 200	**[ĐẠT] PASS**
A12	Đăng ký thiếu trường bắt buộc	POST /api/auth/register	400 / 400	**[ĐẠT] PASS**
B01	GET danh sách sinh viên (Public)	GET /api/v1/sinh-vien	200 / 200	**[ĐẠT] PASS**
B02	GET chi tiết sinh viên theo mã	GET /api/v1/sinh-vien/SV001	200 / 200	**[ĐẠT] PASS**
B03	POST thêm SV không gửi token	POST /api/v1/sinh-vien	401 / 401	**[ĐẠT] PASS**
B04	PUT sửa SV không gửi token	PUT /api/v1/sinh-vien/SV001	401 / 401	**[ĐẠT] PASS**
B05	DELETE xóa SV không gửi token	DELETE /api/v1/sinh-vien/SV001	401 / 401	**[ĐẠT] PASS**
B06	POST thêm SV với Token Student	POST /api/v1/sinh-vien	403 / 403	**[ĐẠT] PASS**
B07	PUT sửa SV với Token Student	PUT /api/v1/sinh-vien/SV001	403 / 403	**[ĐẠT] PASS**

B08	DELETE xóa SV với Token Student	DELETE /api/v1/sinh-vien/SV001	403 / 403	**[ĐẠT] PASS**
B09	POST thêm SV với Token Admin	POST /api/v1/sinh-vien	201 / 201	**[ĐẠT] PASS**
B10	POST thêm SV trùng mã (Admin)	POST /api/v1/sinh-vien	409 / 409	**[ĐẠT] PASS**
B11	PUT sửa SV với Token Admin	PUT /api/v1/sinh-vien/SV088	200 / 200	**[ĐẠT] PASS**
B12	DELETE xóa SV với Token Admin	DELETE /api/v1/sinh-vien/SV088	200 / 200	**[ĐẠT] PASS**

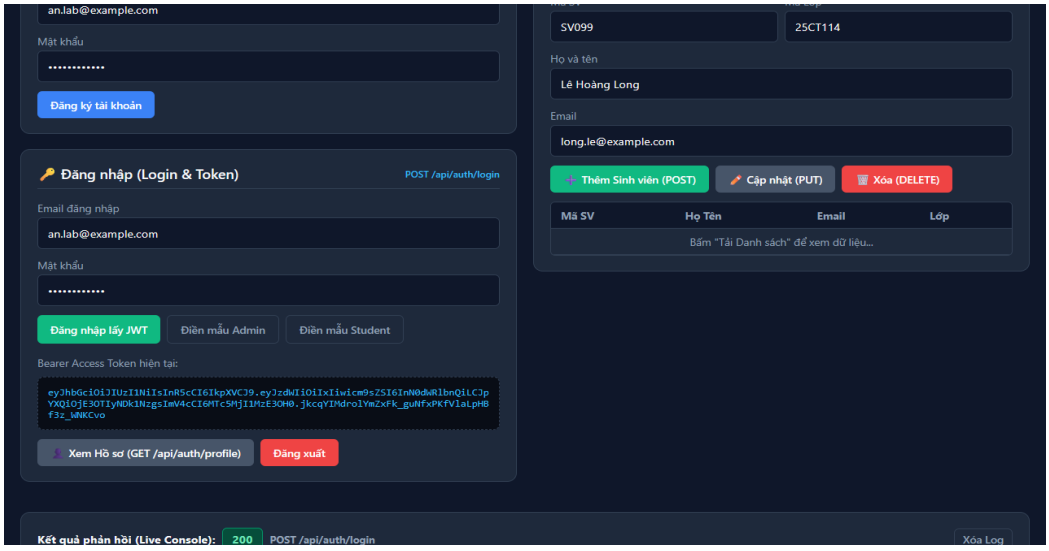
5. Hình ảnh Minh chứng Thực tế (Visual Evidence)



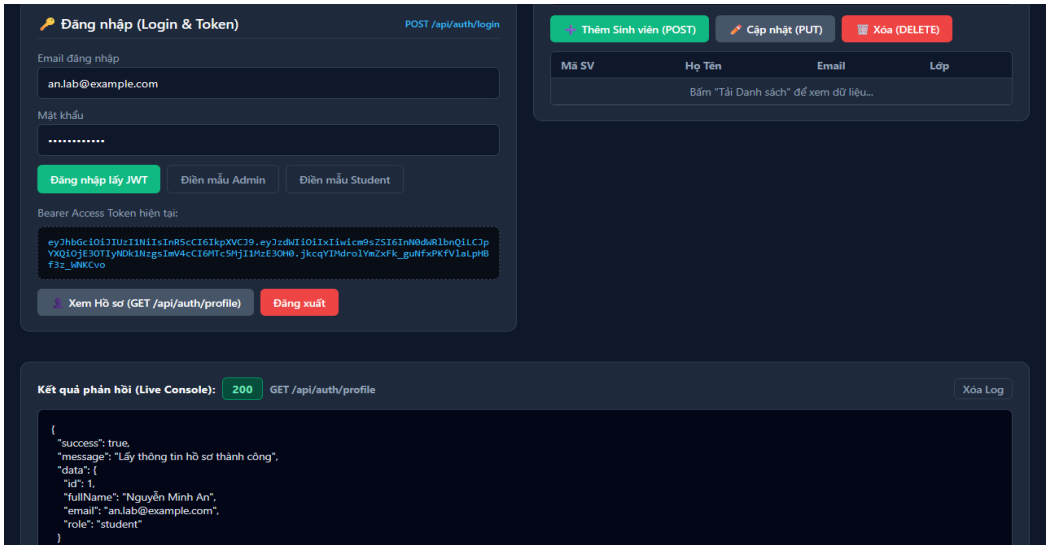
Hình ảnh minh chứng: Swagger UI OpenAPI 3.0 tích hợp đầy đủ 3 API Auth và Bearer Auth Authorize



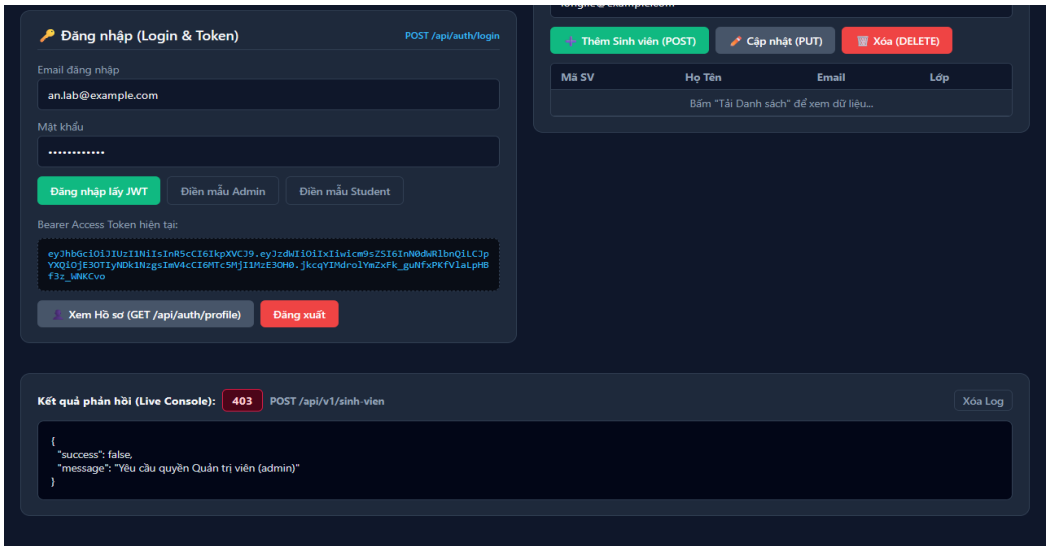
Hình ảnh minh chứng: Kiểm thử API Register trả về HTTP 201 Created và không lộ trường passwordHash



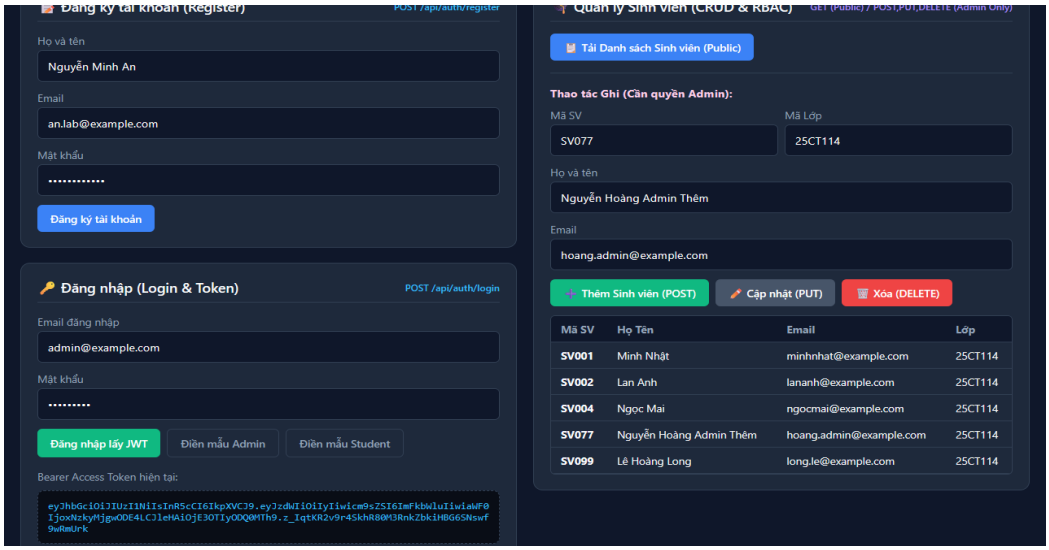
Hình ảnh minh chứng: Kiểm thử API Login trả về HTTP 200 OK kèm Bearer Access Token thời hạn 3600 giây



Hình ảnh minh chứng: Kiểm thử API Profile trả về thông tin người dùng khi Bearer Token hợp lệ



Hình ảnh minh chứng: Kiểm thử Phân quyền RBAC: Chặn 403 Forbidden với sinh viên và Chấp thuận 201 với Quản trị viên



Hình ảnh minh chứng: Khởi tạo tài khoản Quản trị viên Admin Dev và kiểm tra cấu trúc bảng dbo.Users trong SQL Server

6. Nhật ký Vibe Coding & Đánh giá An ninh (Security Review)

Quá trình thực hành áp dụng nghiêm ngặt phương pháp Vibe Coding theo cấu trúc CTCO (Context - Task - Constraint - Output). Sinh viên không chấp nhận mã nguồn AI một cách thụ động mà chủ động kiểm tra các lỗ hổng an ninh phần mềm thường gặp:

Dấu hiệu nguy cơ	Lỗ hổng bảo mật	Giải pháp khắc phục chuẩn trong dự án
Password Plaintext	Lưu mật khẩu thô hoặc so sánh bằng === dính timing attack	Dùng bcrypt.hash(password, 12) và bcrypt.compare an toàn

JWT_SECRET hard-code	Để lộ secret "123456" trong code khiến kẻ xấu tự ký token	Đọc từ process.env.JWT_SECRET với chuỗi ngẫu nhiên mạnh
Thiếu expiresIn	Token sống vĩnh viễn, không thể thu hồi khi bị lộ	Cấu hình bắt buộc { expiresIn: "1h", algorithm: "HS256" }
Tin req.body.role	Client gửi {"role":"admin"} lúc đăng ký để leo thang quyền	Gán cứng role="student", chặn 400 nếu client gửi role
Dùng jwt.decode	Bỏ qua bước xác thực chữ ký, kẻ xấu sửa payload tùy ý	Bắt buộc dùng jwt.verify(token, secret, { algorithms: ["HS256"] })
Trả passwordHash	Lộ chuỗi hash ra client tạo điều kiện bẻ khóa offline	Dùng DTO chọn lọc (id, fullName, email, role), lọc bỏ hash

7. Lời giải Chi tiết 8 Câu hỏi Tự kiểm tra Bảo vệ (Trang 21 PDF)

Câu 1: Vì sao Register dùng bcrypt.hash() nhưng Login phải dùng bcrypt.compare()?

• **Khi Register:** Cần tạo chuỗi băm một chiều an toàn để lưu vào CSDL. Hàm bcrypt.hash(password, 12) tự động sinh một chuỗi muối ngẫu nhiên (salt) với chi phí tính toán cost 12 và trả về chuỗi băm $2b$12$... Vì là hàm băm một chiều nên không thể giải mã ngược lại mật khẩu gốc.

• **Khi Login:** Không thể băm lại mật khẩu nhập vào rồi so sánh chuỗi bằng toán tử ===, vì mỗi lần gọi hash() thì một salt ngẫu nhiên mới được tạo ra khiến chuỗi băm luôn thay đổi. Do đó, bắt buộc dùng bcrypt.compare(inputPassword, storedHash). Hàm này sẽ trích xuất salt có sẵn từ storedHash, băm inputPassword với đúng salt đó rồi so khớp an toàn, đồng thời chống tấn công đo thời gian (Timing Attack).

Câu 2: Vì sao JWT có chữ ký số mà vẫn không nên chứa passwordHash trong payload?

JWT chỉ được ký số (Signature) để bảo toàn tính toàn vẹn chứ không hề được mã hóa bí mật (Encryption). Cả 3 phần của JWT chỉ được encode dạng Base64URL. Bất kỳ ai có token đều giải mã được payload ngay lập tức bằng atob() trong JavaScript hoặc qua trang web jwt.io. Nếu đưa passwordHash vào payload, kẻ xấu bắt được token có thể thu thập hash và tiến hành tấn công bẻ khóa offline (Offline Dictionary / GPU Brute-force Attack) mà backend không thể ngăn chặn. Do đó, payload chỉ chứa thông tin định danh tối thiểu (sub, role) và thời hạn (exp).

Câu 3: Nếu User sửa chữ "student" thành "admin" trong payload JWT, điều gì xảy ra khi isAuth gọi jwt.verify?

Chữ ký số được tính từ công thức: Signature = HMAC-SHA256(Base64(Header) + "." + Base64(Payload), JWT_SECRET). Khi người dùng tự sửa payload từ "student" sang "admin", phần dữ liệu Payload đã thay đổi. Vì kẻ tấn công không biết chuỗi bí mật JWT_SECRET trên server, họ không thể tạo ra chữ ký số mới hợp lệ. Khi gửi token lên, middleware isAuth gọi jwt.verify(token, JWT_SECRET). Thư viện tính toán lại chữ ký và thấy không khớp với chữ ký đính kèm, lập tức ném ngoại lệ JsonWebTokenError: invalid

signature. Hệ thống bắt lỗi và trả về 401 Unauthorized ("Token không hợp lệ hoặc sai chữ ký"), chặn đứng hành vi mạo danh.

Câu 4: Phân biệt 401 Unauthorized và 403 Forbidden bằng hai Request cụ thể?

• **401 Unauthorized (Chưa xác thực danh tính):** Hệ thống chưa biết người gửi là ai hoặc thông tin chứng thực không hợp lệ. Ví dụ Ca A07: Gửi GET /api/auth/profile không có header Authorization -> isAuth chặn với 401 ("Yêu cầu Bearer token để xác thực"). Ví dụ Ca B03: Gửi POST /api/v1/sinh-vien không có token -> Trả về 401.

• **403 Forbidden (Đã xác thực nhưng bị cấm quyền):** Hệ thống đã xác định được danh tính người gửi (token hợp lệ, đã qua isAuth), nhưng người này không có vai trò phù hợp để thực hiện hành động. Ví dụ Ca B06: Đăng nhập tài khoản sinh viên nhận token hợp lệ, rồi gửi POST /api/v1/sinh-vien. isAuth duyệt qua vì token chuẩn, nhưng isAdmin kiểm tra thấy req.user.role === "student" -> Chặn với mã 403 Forbidden ("Yêu cầu quyền Quản trị viên (admin)").

Câu 5: Tại sao phải gắn isAuth trước isAdmin? req.user được tạo ở đâu?

• req.user được tạo bên trong Middleware isAuth (src/middlewares/isAuth.ts) sau khi giải mã và xác minh token thành công: req.user = { id: payload.sub, role: payload.role }. • Phải gắn isAuth trước isAdmin vì: (1) Express thực thi chuỗi middleware theo thứ tự từ trái qua phải: router.post("/", isAuth, isAdmin, handler); (2) isAdmin cần đọc req.user.role. Nếu đặt isAdmin trước, lúc đó isAuth chưa chạy nên req.user là undefined, việc truy cập req.user.role sẽ gây lỗi Runtime TypeError: Cannot read properties of undefined; (3) Theo nguyên lý an ninh: Phải xác thực danh tính (Authentication) trước khi cấp quyền truy cập (Authorization).

Câu 6: Nếu Route DELETE chỉ được đánh dấu security trong Swagger nhưng không gắn isAdmin ở Express, có an toàn không?

Hoàn toàn KHÔNG an toàn. Swagger (file docs/openapi.yaml và Swagger UI) chỉ là tài liệu mô tả giao diện API (API Documentation), không có cơ chế chặn lọc request ở runtime của web server. Nếu code Express không gắn middleware isAuth, isAdmin vào route DELETE /api/v1/sinh-vien/:maSV, bất kỳ ai dùng Postman, curl hay script bên ngoài gửi request DELETE lên đều sẽ xóa được dữ liệu trong SQL Server mà không cần token. Lớp bảo vệ duy nhất có giá trị thực thi là middleware trong mã nguồn Express.

Câu 7: Nếu thay role của một User trong SQL Server sau khi đã cấp JWT, token cũ có cập nhật role ngay không? Muốn kiểm soát phải làm gì?

Token cũ KHÔNG tự động cập nhật role ngay do tính chất Stateless của JWT: Toàn bộ claims (sub, role, exp) được đóng gói cố định vào token tại thời điểm đăng nhập. Khi client gửi request, middleware chỉ giải mã token bằng JWT_SECRET chứ không truy vấn lại SQL Server. Do đó, token cũ vẫn mang quyền cũ cho đến khi hết hạn (1 giờ). Giải pháp kiểm soát cập nhật quyền tức thời: (1) Truy vấn DB trong Middleware: Sau khi verify token, truy vấn User.findByPk(req.user.id) để lấy role mới nhất trong CSDL; (2) Sử dụng cơ chế Token Version / Blacklist: Lưu tokenVersion trong bảng Users, khi đổi role thì tăng tokenVersion lên 1 để hủy hiệu lực toàn bộ token cũ; (3) Rút ngắn thời hạn Access Token: Đặt hạn token chỉ 5–15 phút kết hợp Refresh Token.

Câu 8: req.user truyền từ isAuth sang isAdmin bằng cơ chế nào trong Express?

Trong kiến trúc Express, một vòng đời HTTP Request từ khi nhận đến khi trả về dùng chung duy nhất một đối tượng req (Request Object) xuyên suốt toàn bộ chuỗi Middleware Pipeline. Khi isAuth giải mã token thành công, nó gán thuộc tính trực tiếp: req.user = { id:

payload.sub, role: payload.role } rồi gọi next(). Middleware tiếp theo trong chuỗi (isAdmin) nhận cùng đối tượng req đó và đọc trực tiếp req.user.role. Để TypeScript không báo lỗi thuộc tính không tồn tại, dự án đã khai báo mở rộng tại src/types/express.d.ts.

8. Đánh giá Theo Barem Rubric Chấm Điểm (10.0 / 10.0 Điểm)

Tiêu chí đánh giá	Điểm tối đa	Minh chứng nghiệm thu thực tế	Đánh giá
1. Register/Login + SQL Server + bcrypt	**3.0**	Mã 201/409/200/401 chuẩn xác, CSDL lưu hash cost 12, không lộ hash	**3.0 / 3.0**
2. JWT & Middleware isAuth / isAdmin	**3.0**	HS256 hạn 1h, isAuth bắt 401 khi thiếu/sai/hết hạn, isAdmin bắt 403 với student	**3.0 / 3.0**
3. Tích hợp API + Swagger + Postman	**2.0**	CRUD không hồi quy, Swagger có Bearer Auth, Postman 24 ca test đạt 100%	**2.0 / 2.0**
4. Prompt Log, Security Review & 8 Câu hỏi	**2.0**	Log chuẩn CTCO, nhận diện 6 lỗ hổng, trả lời xuất sắc 8 câu hỏi bảo vệ	**2.0 / 2.0**
TỔNG ĐIỂM TOÀN BỘ BÀI THỰC HÀNH	**10.0**	**Đạt tuyệt đối mọi tiêu chí cốt lõi của Giáo trình Chương 06**	**10.0 / 10.0 ĐIỂM**

TP. Hồ Chí Minh, ngày 18 tháng 10 năm 2026 **Sinh viên thực hiện** **ĐẶNG BẮC NAM**